

Golf Cart

Bring-up progress — sensors, two-host system, vehicle interface, localization

NEWSLab NTU · Autoware LSV meeting · August 2026

The vehicle



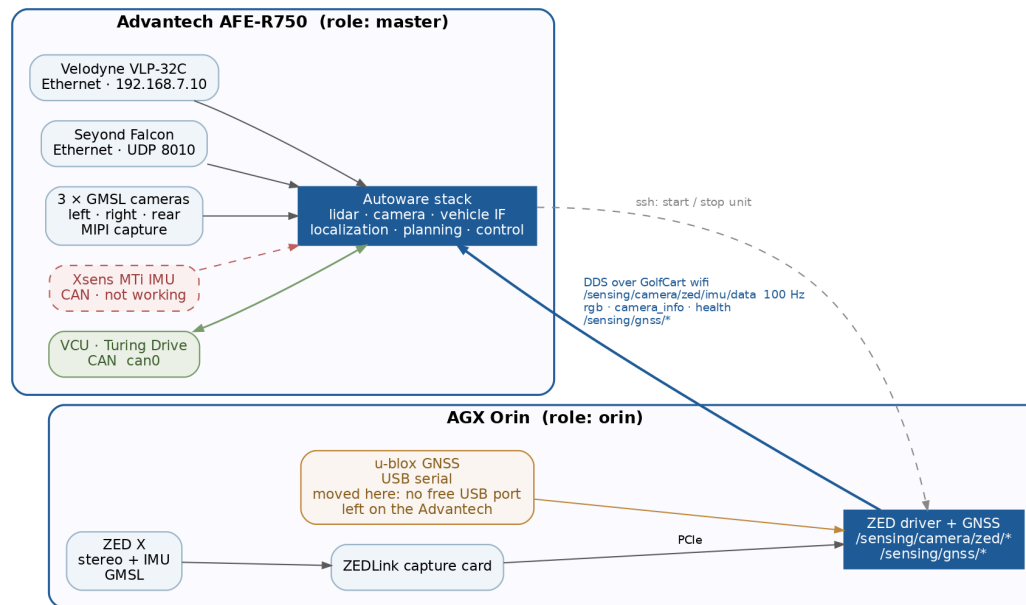
A low-speed campus vehicle on a **Turing Drive VCU**, with an Autoware stack across **two machines**.

Bring-up in five steps:

sensors · two-host system · vehicle interface ·
data collection · autonomous run

Running at NTU today. The 華夏科大 campus is the destination.

Where we are



sensors	ready	two-host	ready	vehicle interface	in progress	data collection	done, data flawed	autonomous run	not started
---------	-------	----------	-------	-------------------	-------------	-----------------	-------------------	----------------	-------------

Sensors: what bit us

All sensors publish. Three things are worth knowing before you copy this.

- **GMSL cameras need a vendor kernel module and a device tree overlay.** The .ko files are ABI-bound to kernel 5.15.148-tegra — a JetPack OTA reinstalls the stock modules and silently undoes it.
- **The Xsens MTi on CAN is not working.** The stack runs on the **ZED X built-in IMU** instead — which sits on the other machine and crosses the network at 100 Hz.
- **The u-blox GNSS is on the Orin**, because the Advantech is short of USB ports. So a second localization input is remote too, for an unrelated reason.

GMSL cameras cost CPU

- Three cameras, **1920×1280 at 30 fps**, emitting **UYVY**.
 - The consuming nodes want **RGB or JPEG**. Something must convert.
 - We tried `nvvidconv`. The conversion still costs us CPU, per camera.
-

A colour conversion per camera, always on, on a box that also carries two LiDARs and the whole Autoware stack.

Fix in progress: `gmslcam`, to take that stage out. The sensor kit migrates to it.

So the machine sits at its limit

This is a fan, held against the cabinet by hand, to keep the box running.

Hence **two machines**:

- compute headroom
- a shared kernel module name the two camera stacks fight over
- ZEDLink only works on the Orin

Splitting the load solved the thermals. It bought a distributed system in exchange.



Launching across two hosts

ROS 2 dropped ROS 1's machine tag. Orchestration is ours. Two problems:

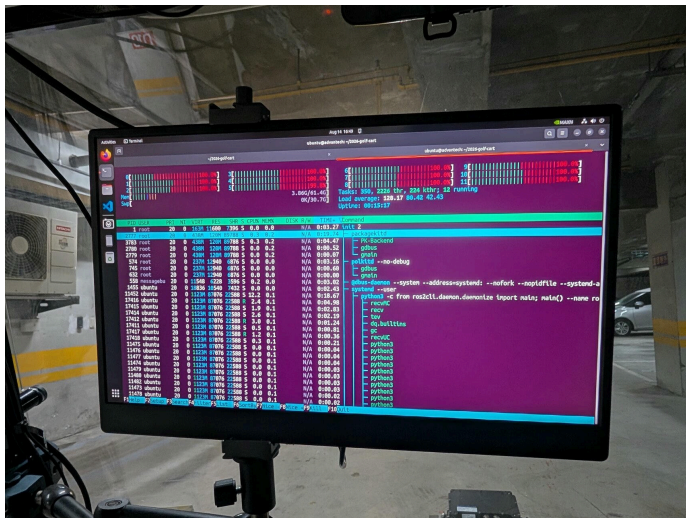
One instance, nothing left behind

- **play_launch** cleans up orphans when the launch dies — and starts the stack in **20 s** against **60 s** for `ros2 launch`
- **systemd user units** around it: singleton by construction, `KillMode=control-group` takes the whole tree down

DDS configured on both sides, then lived with

- one CycloneDDS profile per role, plus a `config/host` marker file
- `scripts/env.sh` reads it, and every shell and every unit sources it
- so a terminal on either box is correct the moment it opens — nobody has to remember which machine they are on

The bill for that speed



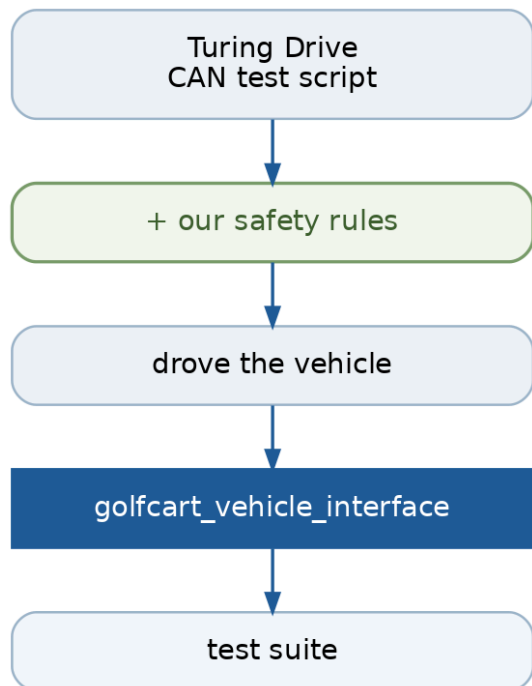
play_launch is fast *because* it spawns as fast as it can. On a box already at its limit, that is a thundering herd — and the machine locks up hard enough to need a power cycle.

Pacing the spawns is the obvious fix. We measured it losing: about 10% fewer runnable tasks for more than double the startup time — spending the exact advantage we adopted the tool for.

What ships is a **1 GiB MemAvailable floor**. Nothing on a healthy boot; holds back only when the machine is about to die.

Gate on the resource that runs out, not on the rate.

Vehicle interface: how it was built



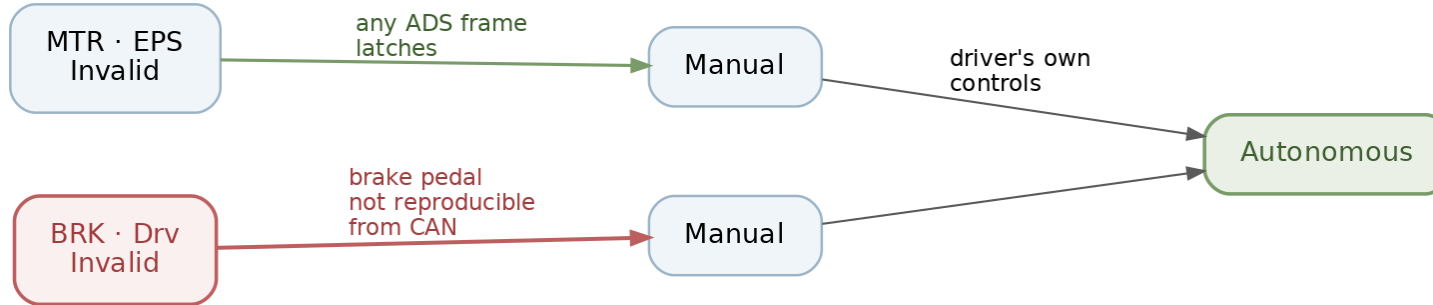
Not written from a specification — grown from the vendor's own test code, and every step exists in the repo.

The safety rules are ours:

- target speed can never go negative — reverse is gear R
- gear P pins speed and angle to zero, re-applied every cycle
- ESTOP release is never a key the terminal cannot send

CAN bindings generate from the vendor DBC at build time — the same file the bench decodes with.

Engage is the VCU's decision — and one door we cannot open



There is no `vehicle_cmd_gate` external selector. The interface commands nothing until all four subsystems report autonomous — **no service call can force it.**

After a VCU restart, **BRK and Drv come up Invalid**, and only a brake-pedal press was seen to clear them. Not reproducible from CAN.

Unattended autonomous start-up is blocked. A vendor question, not more software on our side.

Data collection

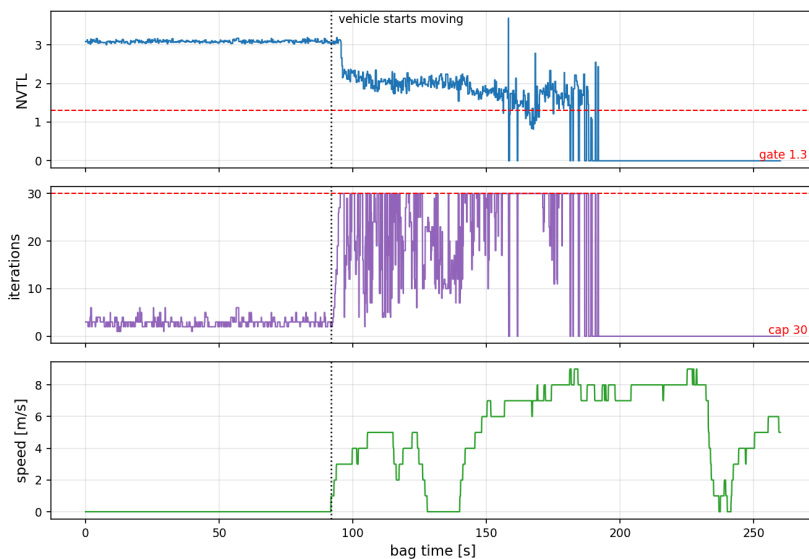


Three runs at NTU. **Both hosts record separately** — each writes the topics for the devices it owns — and the bags merge afterwards.

Only **first-hand driver output** is recorded. Anything a node computed is left out, so replay recomputes it with today's parameters rather than the ones frozen at record time.

Replay is one command: `just ntu-test run` — bag paused for `/clock`, `stack`, `RViz`, initial pose, in the only order that works.

NDT: tuned, and it still breaks



We tuned it and measured it on **scan-to-map residual**, not on the NVTL score — the change that helped accuracy most actually **lowered** NVTL.

It converges parked, to 0.14 m. It **degrades a few seconds after the vehicle starts moving**.

The cause is upstream of NDT, in the recording: 45% of scans are fragments of a revolution, and scans arrive **376 ms stale**.

No raw LiDAR packets were recorded, so these bags cannot be re-decoded. Next run records them.

Status

Step	State	What stands in the way
Sensors	ready	fragmented LiDAR scans; the Xsens IMU is dead
Two-host system	ready	—
Vehicle interface	in progress	the VCU will not enter its autonomous state from CAN alone
Data collection	done, data flawed	the bags carry the sensor defects above
Autonomous run	not started	gated on the vehicle interface blocker

Next

- **Ask Turing Drive what clears the Invalid brake state after a restart.** Everything else on the autonomous run waits behind this.
- **Record raw LiDAR packets**, and check the VLP-32C rotation configuration against the hardware. Then re-run the localization work on data that is not already broken.
- **Migrate the sensor kit to gmslcam**, and take the per-camera CPU conversion out.

TSN is a separate track — happy to cover it if there is interest.